Design and Analysis of Algorithms

Vandana M L
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Introduction to Algorithms, Design Techniques and Analysis

Slides courtesy of **Anany Levitin**

Vandana M L
Department of Computer Science & Engineering

Design and Analysis of Algorithms

Text Books

Book Type	Code	Title & Author	Publication Information		
			Edition	Publisher	Year
Text Book	T1	"Introduction to the Design and Analysis of Algorithms", Anany Levitin, Pearson Education, Delhi (Indian Version)	3	Pearson Education	2012
Reference Book	R1	Introduction to Algorithms Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein	3	Prentice-Hall India	2009
Reference Book	R2	Fundamentals of Computer Algorithms Horowitz, Sahni, Rajasekaran,	2	Universities Press	2007
Reference Book	R3	Algorithm Design Jon Kleinberg, Eva Tardos,	1	Pearson Education	2006

Design and Analysis of Algorithms

Syllabus

- **UNIT I (14 Hours)**
  - Introduction, Fundamentals of Algorithmic Problem solving, Problem Types
  - Analysis of Algorithm Efficiency,
  - Brute Force
- **UNIT II (14 Hours)**
  - Decrease-and-Conquer
  - Divide-and-Conquer
- **UNIT III (14 Hours)**
  - Transform-and-Conquer
  - Space and Time Tradeoffs
  - Greedy Technique
- **UNIT IV (14 Hours)**
  - Limitations of Algorithm Power
  - Coping with the Limitations of Algorithm Power
  - Dynamic Programming

Design and Analysis of Algorithms

Evaluation Policy

Pattern	ISA : 50 Marks ESA : 50 Marks
ISA1 / ISA 2	Hybrid Mode conducted for 40 Marks, Scaled down to 20 Marks each 1 Mark MCQs : 16 questions 2 Mark MCQs : 4 questions 4 Marks Descriptive : 4 questions Total : 40 Marks
Lab Component	NA
Assignment	Banana Problem: Assignment 1 and Assignment 2 From Unit 1 and Unit 2 (In class implementation on Hacker rank + Viva): 10 Marks. Orange Problem: Assignment 3 and Assignment 4 from Unit 3 and Unit 4 (In class implementation on Hacker rank+ Viva): 20 Marks. Jack Fruit Problem : Group Assignment : offline Implementation, Written and Demonstration : 30 Marks Total 60 Marks scaled down to 10
ESA	25 marks from each unit conducted for 100 Marks scaled down to 50 Marks

What is an algorithm?

An algorithm is a sequence of unambiguous instructions for solving a problem,

i.e., for obtaining a required output for any legitimate input in a finite amount of time

Important Points about Algorithms

- The non-ambiguity requirement for each step of an algorithm cannot be compromised
- The range of inputs for which an algorithm works has to be specified carefully.
- The same algorithm can be implemented in several different ways
- There may exist several algorithms for solving the same problem.

Characteristics of Algorithm

Input: Zero or more quantities are externally supplied

Definiteness: Each instruction is clear and unambiguous

Finiteness: The algorithm terminates in a finite number of steps.

Effectiveness: Each instruction must be primitive and feasible

Output: At least one quantity is produced

Algorithm

Why do we need Algorithms?

- It is a tool for solving well-specified Computational Problem.
- Problem statement specifies in general terms relation between input and output
- Algorithm describes computational procedure for achieving input/output relationship
  This Procedure is irrespective of implementation details

Why do we need to study algorithms?

Exposure to different algorithms for solving various problems helps develop skills to design algorithms for the problems for which there are no published algorithms to solve it

Design and Analysis of Algorithms

Basic Issues related to Algorithms

- How to design algorithms
- How to express algorithms
- Proving correctness of designed algorithm
- Efficiency
  - Theoretical analysis
  - Empirical analysis

Basic Issues related to Algorithms

- How to design algorithms
- How to express algorithms
- Proving correctness of designed algorithm
- Efficiency
  - Theoretical analysis
  - Empirical analysis

Algorithm Design Technique

What do you mean by Algorithm Design Techniques?

General Approach to solving problems algorithmically .
Applicable to a variety of problems from different areas of computing

Various Algorithm Design Techniques

- Brute Force
- Divide and Conquer
- Decrease and Conquer
- Transform and Conquer
- Dynamic Programming
- Greedy Technique
- Branch and Bound
- Backtracking

Importance Framework for designing and analyzing algorithms for new problems

Design and Analysis of Algorithms

Basic Issues related to Algorithms

- How to design algorithms
- How to express algorithms
- Proving correctness of designed algorithm
- Efficiency
  - Theoretical analysis
  - Empirical analysis

Methods of Specifying an Algorithm

- **Natural language**
  - Ambiguous
- **Pseudocode**
  - A mixture of a natural language and programming language-like structures
  - Precise and succinct.
  - Pseudocode in this course
    - omits declarations of variables
    - use indentation to show the scope of such statements as for, if, and while.
    - use ← for assignment
- **Flowchart**
  - Method of expressing algorithm by collection of connected geometric shapes

Euclid's Algorithm

Problem: Find gcd(m,n), the greatest common divisor of two nonnegative, not both zero integers m and n

Examples: gcd(60,24) = 12, gcd(60,0) = 60, gcd(0,0) = ?

Euclid's algorithm is based on repeated application of equality

gcd(m,n) = gcd(n, m mod n)

until the second number becomes 0, which makes the problem trivial.

Example: gcd(60,24) = gcd(24,12) = gcd(12,0) = 12

Methods of Specifying an Algorithm

Two descriptions of Euclid's algorithm

Euclid's algorithm for computing gcd(m,n)

Step 1 If n = 0, return m and stop; otherwise go to Step 2

Step 2 Divide m by n and assign the value of the remainder to r

Step 3 Assign the value of n to m and the value of r to n. Go to step 1.

ALGORITHM Euclid(m,n)

//computes gcd(m,n) by Euclid's method

//Input: Two nonnegative,not both zero integers

//Output:Greatest common divisor of m and n

while n ≠ 0 do

    r ← m mod n

    m← n

    n ← r

return m

Algorithm for Sequential Search

ALGORITHM *SequentialSearch*$(A[0..n-1], K)$

```
//Searches for a given value in a given array by sequential search
//Input: An array A[0..n − 1] and a search key K
//Output: The index of the first element of A that matches K
//        or −1 if there are no matching elements
i ← 0
while i < n and A[i] ≠ K do
    i ← i + 1
if i < n return i
else return −1
```

Basic Issues related to Algorithms

- How to design algorithms
- How to express algorithms
- Proving correctness of designed algorithm
- Efficiency
  - Theoretical analysis
  - Empirical analysis

Design and Analysis of Algorithms

Basic Issues related to Algorithms

- How to design algorithms
- How to express algorithms
- Proving correctness of designed algorithm
- Efficiency
  - Theoretical analysis
  - Empirical analysis

Need of Analysis

- To determine resource consumption
  - CPU time
  - Memory space
- Compare different methods for solving the same problem before actually implementing them and running the programs.
- To find an efficient algorithm for solving the problem

Complexity of an algorithm

- A measure of the performance of an algorithm

- An algorithm's performance is characterized by

  - Time complexity

    How fast an algorithm maps input to output as a function of input

  - Space complexity

    amount of memory units required by the algorithm in addition to the memory needed for its input and output

How to determine complexity of an algorithm?

- Experimental study(Performance Measurement)
- Theoretical Analysis (Performance Analysis)

Limitations of Performance Measurement

- It is necessary to implement the algorithm, which may be difficult
- Results may not be indicative of the running time on other inputs not included in the experiment.
- In order to compare two algorithms, the same hardware and software environments must be used
- Experimental data though important is not sufficient

Performance Analysis

- Uses a high-level description of the algorithm instead of an implementation
- Characterizes running time as a function of the input size, n.
- Takes into account all possible inputs
- Allows us to evaluate the speed of an algorithm independent of the hardware/software environment

Multiple choice Questions

Which of the following statement best explains why finiteness is a necessary characteristic of an algorithm ?

a) It ensures that the algorithm has at least one output
b) It guarantees algorithm can handle all the possible inputs
c) It ensures that the algorithm terminates after a finite number of steps
d) It ensures that the algorithm is independent of implementation details

Answer

c) It ensures that the algorithm terminates after a finite number of steps

Multiple choice Questions

An algorithm step states: *"Process the data appropriately and update the result."* This step makes the algorithm unclear and open to interpretation. Which fundamental property of algorithms is violated?

a) Finiteness
b) Definiteness
c) Effectiveness
d) Correctness

Answer

c) Definiteness

Multiple choice Questions

Which of the following best justifies the statement *"The same algorithm can be implemented in several different ways"?*

a) Algorithms are always written in pseudocode

b) Algorithms describe logic independent of programming language and hardware

c) Algorithms must always use recursion

d) Algorithms are optimized only during implementation

Answer

b) Algorithms describe logic independent of programming language and hardware

Multiple choice Questions

Given a problem for which no known algorithm exists, studying existing algorithms primarily helps because it:

a) Allows direct reuse of code
b)Eliminates the need for correctness proofs
c) Develops skills to design new algorithms using known techniques
d) Guarantees optimal efficiency

Answer

c) Develops skills to design new algorithms using known techniques

Multiple choice Questions

Which pair of issues is most closely related when evaluating the *efficiency* of an algorithm?

a) Correctness and definiteness

b) Input specification and output generation

c) Theoretical analysis and empirical analysis

d) Design technique and expression method

Answer

c) Theoretical analysis and empirical analysis

Multiple choice Questions

Which of the following distinguishes pseudocode (as used in this course) from a programming language?

a) It does not allow conditional statements

b) It omits variable declarations and uses indentation to define scope

c) It cannot express iterative constructs

d) It cannot be translated into executable code

Answer

b) It omits variable declarations and uses indentation to define scope

Multiple choice Questions

In Euclid's algorithm, the correctness fundamentally relies on which mathematical property?

a) gcd(m, n) = gcd(m — n, n)

b) gcd(m, n) = gcd(n, m mod n)

c) gcd(m, n) = gcd(m / n, n)

d) gcd(m, n) = m n

Answer

b) gcd(m, n) = gcd(n, m mod n)

What happens if Euclid's algorithm is applied to the input pair (0, 0)?

a) The algorithm returns 0

b) The algorithm returns 1

c) The algorithm enters an infinite loop

d) The problem is undefined and violates the input constraint

Answer

d) The problem is undefined and violates the input constraint

Multiple choice Questions

Which algorithm design technique is most appropriate when a problem can be solved by making a locally optimal choice at each step, with no need to reconsider earlier decisions?

a) Dynamic Programming

b) Divide and Conquer

c) Greedy Technique

d) Backtracking

Answer

c) Greedy Technique

Multiple choice Questions

Why is Theoretical Analysis Preferred over experimental performance measurement when comparing two algorithms ?

a) Experimental Analysis is inaccurate

b) Theoretical Analysis ignores hardware constraints

c) Experimental Analysis cannot be repeated

d) Theoretical Analysis evaluates performance independent of hardware and software

Answer

d) Theoretical Analysis evaluates performance independent of hardware and software

Multiple choice Questions

Which of the following is a key limitation of experimental performance measurement

a) It cannot measure memory usage

b) It requires pseudocode instead of implementation

c) Results may not generalize to inputs not included in the experiments

d) It always over estimates running time

Answer

c) Results may not generalize to inputs not included in the experiments

THANK YOU

Vandana M L
Department of Computer Science & Engineering

vandanamd@pes.edu